

ARCHITECTURE MICRO-SERVICES

Polycopié étudiant — Journée 1

Séances S1 · S2 · S3 · S4

De l'introduction au premier service

Définitions · Décomposition · API REST · TP Django

≈ 7 heures de cours et pratique

Sommaire de la journée

Avant tout

- Dictionnaire complet de la journée (à garder sous les yeux)

S1 — Introduction aux micro-services

- Architecture logicielle, monolithe, SOA
- Définition micro-services (Lewis & Fowler)
- Historique : Bezos, Netflix, 2014
- Avantages et inconvénients
- Quand utiliser, quand surtout pas

S2 — Décomposition en services

- Pourquoi le découpage est LE problème dur
- Domain-Driven Design et bounded contexts
- 6 stratégies de décomposition
- Loi de Conway et Reverse Conway
- Anti-patterns et atelier de découpage

S3 — Conception des API REST

- Pourquoi REST a gagné
- Ressources et URIs propres
- Verbes HTTP, idempotence, codes de statut
- Pagination, filtrage, tri, versioning
- Erreurs structurées et documentation OpenAPI

S4 — TP : votre premier micro-service Django

- Initialisation projet Django + DRF
- Modèle, serializer, ViewSet, router
- Pagination, filtrage, recherche
- Tests unitaires, Postman, OpenAPI
- Git, README, livraison

Pour finir

- Devoir global de la journée à rendre la semaine suivante

Dictionnaire complet de la journée

Ce dictionnaire regroupe TOUS les termes anglais, abréviations et notions techniques que tu vas rencontrer pendant les 4 séances. Il est organisé en 4 sections pour aider à la mémorisation.

Tu peux y revenir à tout moment pendant les cours et le TP.

A — Architecture & micro-services (S1)

Terme	Définition
Architecture logicielle	Ensemble des décisions structurantes définissant comment un système est découpé, comment ses parties communiquent et comment elles évoluent.
Monolithe	Application déployée comme une seule unité, dont tout le code partage un même processus et généralement une seule base de données.
Micro-service	Composant applicatif petit, autonome, déployable indépendamment, qui implémente une capacité métier précise et communique par le réseau.
SOA	Service-Oriented Architecture. Ancêtre des micro-services, années 2000, reposant sur un bus d'entreprise (ESB).
ESB	Enterprise Service Bus. Composant central qui orchestre la communication entre services dans la SOA classique.
Bezos Mandate	Note interne de Jeff Bezos en 2002 imposant le découpage en services chez Amazon. Acte de naissance silencieux des micro-services.
Scalabilité	Capacité d'un système à absorber une charge croissante. Horizontale = ajouter des machines, verticale = machine plus puissante.
Déploiement indépendant	Capacité à mettre en production un service sans redéploier les autres. LA caractéristique clé des micro-services.
Résilience	Capacité à continuer à fonctionner partiellement quand une partie est en panne.
Couplage	Degré de dépendance entre composants. Faible couplage = composants évolutifs séparément.
Cohésion	Degré d'unité interne d'un composant. Forte cohésion = toutes les fonctions servent le même but.
Polyglottisme	Chaque service peut être écrit dans un langage et avec une techno différente.
You build it, you run it	Principe : l'équipe qui code un service est aussi responsable de son fonctionnement en production.

Terme	Définition
Eventual consistency	Cohérence à terme. Les données peuvent être temporairement désynchronisées entre services mais convergent à terme.
Distributed monolith	Anti-pattern : services techniquement séparés mais si couplés qu'on ne peut pas les déployer indépendamment.
Two-pizza team	Équipe assez petite pour être nourrie par deux pizzas (~6-8 personnes), principe Amazon.
MVP	Minimum Viable Product. Première version d'un produit qui cherche son marché.
Product-market fit	Moment où un produit trouve un marché qui en veut vraiment.
Monolith first	Principe de Fowler : commencer monolithique modulaire, extraire en micro-services seulement ce qui le justifie.

B — DDD & décomposition (S2)

Terme	Définition
DDD	Domain-Driven Design. Approche d'Eric Evans (2003) qui place le métier au centre de la conception.
Domain (domaine)	L'ensemble du métier que ton logiciel doit servir. Ex : e-commerce, banque.
Sous-domaine	Une partie du domaine, avec son vocabulaire propre.
Bounded context	Frontière logique d'un sous-domaine où un modèle et un vocabulaire ont un sens précis. Concept central du DDD.
Ubiquitous language	Langage commun entre développeurs et experts métier dans un bounded context.
Capacité métier	Une chose que le métier fait. Bonne unité de découpage en services.
Loi de Conway	« Toute organisation qui conçoit un système produit un système dont la structure copie la structure de communication de l'organisation. » (1968)
Reverse Conway	Réorganiser les équipes EN PREMIER pour que l'architecture cible émerge.
Event Storming	Atelier collaboratif où on identifie les événements métier pour en déduire les bounded contexts.
Strangler fig pattern	Patron pour migrer un monolithe : on extrait progressivement des services jusqu'à ce que le monolithe disparaisse.
Anti-corruption layer	Couche tampon qui protège un service d'un autre dont le modèle est différent ou incompatible.

Terme	Définition
Granularité	Taille des services. Trop fine = nano-services. Trop grosse = monolithes déguisés.
God service	Anti-pattern : service trop gros qui devient le hub central. Goulot d'étranglement.
Chatty service	Anti-pattern : service qui doit appeler trop d'autres pour faire son travail.
Entity service	Anti-pattern : un service par table SQL plutôt que par capacité métier.
Shared database	Anti-pattern : plusieurs services partagent la même base. Plus d'indépendance.

C — HTTP & REST (S3)

Terme	Définition
API	Application Programming Interface. Contrat permettant à un logiciel d'utiliser un autre.
REST	Representational State Transfer. Style d'architecture pour API web défini par Roy Fielding en 2000.
Ressource	Une chose métier exposée par l'API. Identifiée par une URI. Ex : un produit, une commande.
URI	Uniform Resource Identifier. L'adresse qui identifie une ressource. Ex : /api/v1/products/42
Verbe HTTP	Méthode HTTP qui définit l'action : GET (lire), POST (créer), PUT (remplacer), PATCH (modifier), DELETE (supprimer).
Code de statut	Nombre à 3 chiffres dans la réponse HTTP. 2xx = succès, 4xx = erreur client, 5xx = erreur serveur.
Idempotence	Propriété d'une opération qui produit le même résultat appelée 1 ou N fois. GET, PUT, DELETE sont idempotents.
Statelessness	Chaque requête contient tout ce qu'il faut pour la traiter. Le serveur ne maintient pas d'état.
Endpoint	Une URI combinée à un verbe HTTP = une opération précise.
Pagination	Découpe d'une grande collection en pages plus petites. Offset/limit ou cursor-based.
Versioning	Mécanisme pour gérer l'évolution d'une API. Par URI (/api/v1/) ou header.
HATEOAS	Hypermedia As The Engine Of Application State. Niveau 3 de REST où la réponse inclut les liens vers les actions possibles.

Terme	Définition
OpenAPI (Swagger)	Standard de documentation d'API REST en YAML/JSON. Permet de générer doc, clients, tests.
Swagger UI	Interface web qui rend une spec OpenAPI navigable et testable directement dans le navigateur.
RFC 7807	Standard pour renvoyer des erreurs structurées (Problem Details for HTTP APIs).
CORS	Cross-Origin Resource Sharing. Mécanisme navigateur qui contrôle quels domaines peuvent appeler une API.
JWT	JSON Web Token. Jeton d'authentification standard en micro-services.
mTLS	Mutual TLS. Chiffrement et authentification mutuelle entre services.
Sagas	Pattern qui gère une transaction métier longue répartie sur plusieurs services.
Tracing distribué	Suivre une requête à travers plusieurs services pour debug (Jaeger, Zipkin, OpenTelemetry).

D — Django & DRF (S4)

Terme	Définition
Projet Django	Container global d'une application Django. Créé par django-admin startproject.
App Django	Module fonctionnel dans un projet. Un projet a 1 à N apps.
Modèle (model)	Classe Python qui représente une table en base. Hérite de models.Model.
Migration	Fichier Python qui décrit une évolution du schéma. Généré par makemigrations, appliqué par migrate.
ORM	Object-Relational Mapping. Traduit code Python ↔ requêtes SQL automatiquement.
QuerySet	Liste paresseuse de résultats Django. filter(), exclude(), order_by()...
DRF	Django REST Framework. Bibliothèque tierce standard pour faire des API REST avec Django.
Serializer	Classe DRF qui transforme modèle ↔ JSON et valide les données.
ModelSerializer	Serializer auto-généré à partir d'un modèle. 90 % des cas d'usage.
ViewSet	Vue DRF qui regroupe plusieurs opérations (list, retrieve, create, update, destroy) en une classe.

Terme	Définition
ModelViewSet	ViewSet auto-généré pour modèle + serializer. CRUD complet en quelques lignes.
Router DRF	Génère automatiquement les URLs d'un ViewSet (/products/, /products/{id}/, etc.).
drf-spectacular	Bibliothèque qui génère le schéma OpenAPI 3 depuis le code DRF.
APITestCase	Classe DRF pour tester les APIs. self.client.get(), .post(), .patch()...
Minor unit (centimes)	Stocker 8990 au lieu de 89.90 €. Approche fintech Stripe/PayPal.
DevOps	Culture qui fusionne développement et opérations (déploiement, monitoring).
Kubernetes (K8s)	Orchestrateur de conteneurs Docker. Outil standard en production.

S1

Introduction aux micro-services

Définitions · Historique · Avantages · Inconvénients

S1 — Objectifs

À l'issue de cette première séance, vous serez capable de :

- Définir ce qu'est une architecture micro-services.
- Énoncer 5 différences concrètes entre une architecture monolithique et une architecture micro-services.
- Citer au moins 4 avantages et 4 inconvénients des micro-services avec un exemple pour chacun.
- Positionner les micro-services dans l'histoire des architectures logicielles.
- Identifier au moins 2 cas d'usage où les micro-services sont pertinents et 2 cas où ils ne le sont pas.

S1.1 — Qu'est-ce qu'une architecture logicielle ?

L'architecture logicielle désigne l'ensemble des décisions structurantes qui définissent comment un système est découpé en parties, comment ces parties communiquent, et comment l'ensemble peut évoluer dans le temps. Ce sont les choix coûteux à changer plus tard.

On peut développer un logiciel sans réfléchir à son architecture, mais on architecture toujours, même par défaut. La question n'est pas « faut-il une architecture ? » mais « quelle architecture choisit-on, et est-ce qu'on l'a choisie consciemment ? ».

Analogie : l'urbanisme

Imagine la construction d'une ville. Tu peux choisir de construire une seule mégastructure géante qui contient tout (logements, commerces, bureaux, école) : c'est le monolithe. Pratique pour circuler à l'intérieur, mais si un étage prend feu, toute la mégastructure est menacée.

Ou tu peux construire une ville avec des bâtiments séparés, reliés par des rues. C'est plus complexe à organiser (il faut des routes, des transports, des règlements), mais chaque bâtiment peut être démoli et reconstruit sans toucher aux autres. C'est l'esprit micro-services.

S1.2 — L'architecture monolithique

Définition

Une architecture monolithique est un style dans lequel toute l'application est packagée et déployée comme une seule unité, exécutée dans un même processus, partageant le plus souvent une base de données unique.

Important : monolithe n'est pas un gros mot

Pendant 30 ans, la quasi-totalité des applications ont été monolithiques. WordPress, Magento, des milliers d'ERP métier sont monolithiques. On distingue le « big ball of mud » (où tout est emmêlé) du « monolithe modulaire » (organisé en modules clairs).

Caractéristiques

- Un seul code base, un seul dépôt Git généralement.
- Un seul processus à l'exécution, avec des appels de fonction internes plutôt qu'HTTP.
- Une seule base de données partagée.
- Un seul artefact de déploiement.
- Mise à l'échelle horizontale grossière : on duplique tout, même les parties peu sollicitées.

Forces

- Simplicité de développement : tout est local, un seul IDE, pas de réseau.
- Simplicité de déploiement : un seul artefact à pousser.
- Transactions ACID natives.
- Debug simple : un seul stack trace.
- Performance : appel de fonction = mille fois plus rapide qu'HTTP.

Limites

- Plus le code grossit, plus il devient difficile à comprendre.
- Une équipe nombreuse a du mal à travailler sans se marcher dessus.
- Il faut tout redéploier pour la moindre modification.
- Mise à l'échelle peu efficace.
- Le moindre crash met toute l'application par terre.

S1.3 — La SOA (Service-Oriented Architecture)

Style apparu dans les années 1990-2000 dans lequel les fonctionnalités sont exposées comme des services réutilisables, coordonnés par un bus d'entreprise (ESB).

Caractéristiques de la SOA classique

- Services exposés via SOAP et WSDL (XML lourd, contrats stricts).
- Bus central (ESB) qui orchestre les échanges.
- Services souvent gros (coarse-grained).
- Gouvernance centralisée.

À retenir

« Les micro-services, c'est de la SOA sans bus centralisé, avec des services plus petits et des équipes plus autonomes. »

S1.4 — L'architecture micro-services

Définition canonique (Lewis & Fowler, 2014)

« Une approche de développement d'une application unique sous la forme d'une suite de petits services, chacun s'exécutant dans son propre processus et communiquant via des mécanismes légers, souvent une API HTTP. Ces services sont construits autour de capacités métier, sont déployables indépendamment par des machines de déploiement entièrement automatisées. »

Décortiquons

- « **Petits services** » : ce qu'on livre est UNE application cohérente, mais en interne plusieurs petits programmes.
- « **Propre processus** » : processus distincts. Si l'un crashe, les autres survivent.
- « **Mécanismes légers** » : HTTP/REST, gRPC, messagerie (Kafka, RabbitMQ).
- « **Capacités métier** » : découpage par fonction métier, pas par couche technique.
- « **Déployables indépendamment** » : LA caractéristique clé.

💡 Analogie : la brigade de restaurant

Un monolithe : un chef cuisinier seul qui prépare toute la commande de A à Z. Simple à coordonner mais ne passe pas l'échelle.

Une architecture micro-services : une brigade. Un chef garde-manger pour les entrées, un saucier, un pâtissier. Chacun travaille en parallèle, avec ses outils. La salle reçoit un repas cohérent, mais la cuisine est plusieurs ateliers spécialisés.

S1.5 — Historique

- **2002** : Bezos Mandate chez Amazon. La note de 5 lignes qui impose le découpage en services. Acte de naissance silencieux.
- **2009-2016** : Netflix migre vers le cloud après une panne majeure. Devient le porte-étendard public.
- **Mai 2011** : le terme « microservice » émerge à un workshop à Venise.
- **Mars 2014** : Lewis & Fowler publient l'article fondateur sur martinfowler.com.
- **2016-2018** : pic de hype, puis maturité raisonnée.

S1.6 — Les avantages

1. Scalabilité indépendante

On peut faire passer à l'échelle uniquement les services qui en ont besoin. Black Friday : Amazon scale massivement le service Recherche, mais pas l'Historique-commandes.

2. Déploiement indépendant

Une équipe peut pousser une nouvelle version sans coordonner avec les autres. Amazon déploie en moyenne toutes les 11 secondes en interne. Impossible en monolithe.

3. Polyglottisme

Chaque service peut être écrit dans le langage et avec la techno la plus adaptée. Python pour la data science, Java pour le transactionnel, Node pour le temps réel.

4. Résilience par isolation

Si un service tombe, les autres continuent. Sur Amazon, si Recommandations est en panne, la page produit s'affiche sans le bloc « Les clients qui ont acheté ceci... ».

5. Équipes autonomes

Une équipe possède un service de bout en bout : code, base, déploiement, monitoring. « You build it, you run it ».

S1.7 — Les inconvénients

Aussi importants à connaître que les avantages. Trop souvent on ne présente que les avantages aux étudiants.

1. Complexité opérationnelle

Au lieu de gérer 1 application, tu en gères 20. 20 services à surveiller, 20 bases, leurs interactions, leurs versions.

2. Latence réseau

Un appel de fonction : ns. Un appel HTTP : ms. Si une requête traverse 8 services, tu accumules 100ms sans rien faire d'utile.

3. Cohérence des données distribuée

Dans un monolithe, une transaction ACID garantit que soit tout se fait, soit rien. En micro-services, chaque service a sa base. Patterns complexes : Saga, Outbox.

4. Debug distribué

Quand une requête échoue, elle a peut-être traversé 5 services. Tracing distribué obligatoire (Jaeger, OpenTelemetry).

5. Sécurité accrue

Chaque service est une porte d'entrée potentielle. mTLS, JWT, gestion de secrets.

Phrase clé à retenir

« Les micro-services échangent une complexité simple, locale, contre une complexité distribuée. Cet échange vaut le coup quand votre produit est mature et que votre équipe l'est aussi. Il est désastreux quand on le fait pour des raisons de mode. »

S1.8 — Quand utiliser les micro-services

Quand c'est une bonne idée

- Grande équipe qui ralentit à cause de la coordination.
- Produit mature dont les frontières métier sont stables.
- Besoin de scalabilité hétérogène.
- Besoin de mises en production fréquentes et indépendantes.
- Organisation mature côté DevOps.

Quand c'est une mauvaise idée

- Petite équipe (moins de 10 personnes).
- Produit MVP qui cherche encore son product-market fit.
- Équipe sans expérience DevOps.
- Pas de besoin réel de scalabilité fine.
- Application interne à faible charge.

Le « monolithe modulaire » : un troisième chemin

Entre le monolithe « plat » et les micro-services existe une option souvent oubliée : le monolithe modulaire, très bien organisé en modules clairement séparés. De nombreux experts recommandent de commencer ainsi et de n'extraire en micro-services que les modules qui le justifient. C'est le pattern « Monolith first ».

S1.9 — À retenir

- Un micro-service est petit, autonome, déployable indépendamment, et implémente une capacité métier précise.
- Les micro-services ne sont pas une révolution mais une évolution de la SOA, popularisée par Lewis & Fowler en 2014.
- Avantage clé n°1 : la scalabilité indépendante. Avantage clé n°2 : le déploiement indépendant.
- Inconvénient majeur : la complexité distribuée.
- Le monolithe n'est pas un gros mot. Commencer par un monolithe modulaire est souvent sage.
- Les micro-services échangent une complexité simple contre une complexité distribuée.

S2

Décomposition en services

Bounded contexts · DDD · Loi de Conway · Anti-patterns

S2 — Objectifs

À l'issue de cette séance, vous serez capable de :

- Expliquer pourquoi le découpage est LE problème dur des micro-services.
- Définir un « bounded context » et donner un exemple métier.
- Citer 6 stratégies de décomposition.
- Énoncer la loi de Conway et expliquer son impact.
- Reconnaître les 7 principaux anti-patterns de découpage.

S2.1 — Pourquoi le découpage est LE problème dur

Aujourd'hui, on attaque la question pratique : COMMENT découper. C'est la décision la plus importante d'un projet micro-services, et la plus difficile à corriger une fois prise.

Un mauvais découpage donne le pire de tous les mondes : la complexité des micro-services sans aucun de leurs bénéfices. C'est ce qu'on appelle un « distributed monolith ».

Pourquoi c'est si dur

- Une fois un service en production avec sa base, le découper différemment coûte des mois.
- Les frontières métier ne sont pas évidentes.
- Les bonnes frontières dépendent du métier ET de l'organisation.
- Il n'existe pas de « bonne réponse » universelle.

La règle d'or à graver

Découpez par capacité métier (« gérer le catalogue », « encaisser un paiement »), JAMAIS par couche technique (« service base de données », « service API »). Le découpage technique réinvente le monolithe distribué.

S2.2 — Domain-Driven Design

Approche d'Eric Evans (2003) qui place le métier au centre du découpage. Référence pour les micro-services.

Concept clé n°1 — Domaine et sous-domaines

Le DOMAINE = tout le métier que ton logiciel doit servir. Pour FlexShop (notre e-commerce) : vendre des produits en ligne.

Il se décompose en SOUS-DOMAINES : Catalogue, Panier, Commande, Paiement, Livraison, Notifications, Comptes clients.

Concept clé n°2 — Le bounded context

Frontière logique à l'intérieur de laquelle un modèle et un vocabulaire ont un sens précis. LE concept le plus important du DDD.

Exemple : le mot « Commande »

Dans le contexte VENTE, une « Commande » contient : items achetés, prix, client, mode de paiement. Elle change d'état : panier → validée → payée.

Dans le contexte LOGISTIQUE, une « Commande » contient : adresses, poids, dimensions, transporteur, n° de tracking. Elle change d'état : préparée → expédiée → livrée.

MÊME MOT, deux modèles très différents. Le DDD dit : c'est OK. Deux bounded contexts, deux modèles.

Concept clé n°3 — Ubiquitous language

À l'intérieur d'un bounded context, développeurs et experts métier doivent parler le MÊME langage.

Concept clé n°4 — Comment identifier les bounded contexts

- Écouter le vocabulaire métier. Quand le même mot a deux sens, c'est une frontière.
- Identifier les expertises distinctes dans l'organisation.
- Atelier EVENT STORMING : lister les événements métier, les regrouper.

S2.3 — Six stratégies de décomposition

1. Par capacité métier (LA référence)

On identifie ce que le métier FAIT et on crée un service par capacité majeure. E-commerce : Catalogue, Panier, Commande, Paiement, Livraison.

2. Par sous-domaine DDD

Variante structurée. Chaque bounded context devient un service.

3. Par cycle de vie de la donnée

Regrouper les fonctions qui manipulent la même donnée.

4. Par profil de charge

Extraire en service les parties à scalabilité très différente. Ex : Recherche-produit séparée du Catalogue.

5. Par criticité

Isoler les fonctions critiques. Ex : Paiement séparé pour audit et sécurité.

6. Par contrainte technologique

Ex : un service Recommandations en Python à côté d'un Catalogue en Java.

La règle pratique

Mélanger 1 (capacité métier) comme structure principale, affiner avec 4, 5, 6.

S2.4 — La loi de Conway

L'énoncé original (Melvin Conway, 1968)

« Toute organisation qui conçoit un système produit nécessairement un système dont la structure est une copie de la structure de communication de l'organisation. »

Autrement dit : ton architecture ressemble TOUJOURS à l'organigramme. Que tu le veuilles ou non.

Cas vécu

Une grande banque avait 4 équipes : Java, .NET, Cobol, Mainframe. Quand elle a voulu migrer vers les micro-services, elle a produit 4 services. Un par techno, pas par métier.

Chaque opération métier nécessitait d'appeler les 4 services. Latence catastrophique. Ils ont dû tout reprendre.

Le « Reverse Conway Maneuver »

Plutôt que de subir la loi de Conway, on l'utilise. On RÉORGANISE LES ÉQUIPES selon l'architecture cible, et l'architecture émerge.

Two-pizza team (Amazon)

Une équipe ne doit pas être plus grande qu'on ne peut nourrir avec deux pizzas. En pratique : 6 à 8 personnes maximum.

S2.5 — Les 7 anti-patterns à éviter

1. Découpage par couches techniques

Service UI, service métier, service base de données. C'est un monolithe distribué.

2. Distributed monolith

Services techniquement séparés mais impossible à déployer indépendamment.

3. Nano-services

Découpage trop fin. Chaque fonction devient un service.

4. God service

Service trop gros que tous les autres doivent appeler.

5. Shared database

Plusieurs services partagent la même base. Plus d'indépendance.

6. Chatty service

Service qui doit appeler 5 ou 10 autres pour faire son travail.

7. Entity service

Un service par TABLE plutôt que par capacité métier.

S2.6 — Atelier de découpage

Exercice concret pratiqué en cours, à refaire chez vous.

Le cas : un système hôtelier

Une chaîne hôtelière veut digitaliser :

- Réservation en ligne, check-in/out, gestion des chambres.
- Facturation, programme fidélité, ménage, restauration.
- Notifications, reporting.

Consignes

- Proposez un découpage en 5 à 8 services.
- Justifiez chaque service par sa capacité métier.
- Listez les interactions entre services.

Une solution possible

Réservation : création, modification, annulation, acompte, disponibilité.

Séjour : check-in/out, gestion sur place, room service.

Housekeeping : état des chambres, ménage, maintenance.

Facturation + Paiement : calcul total, TVA, encaissement (Paiement isolé pour la sécurité).

Fidélité, Notifications, Reporting : logiques propres, charges différentes.

S2.7 — À retenir

- Le découpage est LE problème dur des micro-services.
- Découper par CAPACITÉ MÉTIER, JAMAIS par couche technique.
- Le DDD donne le bon outillage : bounded contexts, ubiquitous language.
- Un bounded context = frontière où un mot a UN sens précis.
- La loi de Conway : ton archi reflètera l'organigramme. Soit tu la subis, soit tu l'utilises.
- Sept anti-patterns à éviter.
- Une équipe par service, max 6-8 personnes.

S3

Conception des API REST

Ressources · Verbes · Codes de statut · Pagination · Versioning · OpenAPI

S3 — Objectifs

À l'issue de cette séance, vous serez capable de :

- Citer les principes de base d'une API REST.
- Choisir le bon verbe HTTP et le bon code de statut pour chaque opération.
- Concevoir des URIs propres et cohérentes.
- Implémenter la pagination, le filtrage et le tri.
- Mettre en place le versioning d'une API.
- Renvoyer des erreurs structurées.
- Documenter une API avec OpenAPI 3.0 et Swagger UI.

S3.1 — Pourquoi REST est devenu LE standard

Quand deux services doivent se parler, ils ont besoin d'un protocole commun. En micro-services, REST est massivement dominant.

Pourquoi cette domination

- REST repose sur HTTP, omniprésent et standardisé.
- Tout langage sait faire des requêtes HTTP.
- Outils matures : Postman, curl, navigateurs.
- Les humains lisent et debuguent facilement.
- Statelessness facilite la scalabilité horizontale.

REST n'est pas un standard, c'est un STYLE

Défini par Roy Fielding en 2000 dans sa thèse. Il y a des règles, mais aussi du jugement. Deux API REST bien faites peuvent avoir des choix différents.

Les 6 contraintes REST

- Architecture client-serveur.
- Stateless.
- Cacheable.
- Interface uniforme.
- Système en couches.
- Code on demand (optionnel).

S3.2 — Penser en ressources, pas en actions

LE point conceptuel le plus important. Et le plus difficile pour les débutants.

Le glissement de mindset

- **Mauvais réflexe** : « j'ai besoin de l'action ANNULER UNE COMMANDE, je crée /cancelOrder. »
- **Bon réflexe** : « j'ai une ressource COMMANDE qui peut changer d'état. POST /orders/42/cancellation. »

Règles de nommage des URIs

- Toujours au PLURIEL pour les collections : /products, /orders, /users.
- Pas de verbes dans les URIs : POST /products, pas /createProduct.
- Hiérarchie logique : /orders/42/items pour les items de la commande 42.
- Tirets plutôt que camelCase : /order-items.
- Versionner : /api/v1/products.

Cas d'usage	✗ Mauvais	✓ Bon
Lister tous les produits	GET /getAllProducts	GET /products
Récupérer le produit 42	GET /product?id=42	GET /products/42
Créer un produit	POST /createProduct	POST /products
Items d'une commande	GET /getOrderItems?orderId=5	GET /orders/5/items
Annuler une commande	POST /cancelOrder?id=5	POST /orders/5/cancellation

S3.3 — Les verbes HTTP

Verbe	Action	Idempotent ?	Exemple
GET	Lire une ressource ou une collection	Oui	GET /products/42
POST	Créer une nouvelle ressource	Non	POST /products
PUT	Remplacer entièrement	Oui	PUT /products/42
PATCH	Modifier partiellement	Pas garanti	PATCH /products/42
DELETE	Supprimer	Oui	DELETE /products/42

PUT vs PATCH

- **PUT** : remplace TOTALEMENT. Tu envoies tous les champs.
- **PATCH** : modifie PARTIELLEMENT. Tu envoies seulement ce qui change.

Pourquoi l'idempotence est cruciale

Une opération idempotente peut être rejouée sans risque. Si une requête échoue (timeout réseau), un retry avec un verbe idempotent est sûr. Avec POST, il créerait 2 ressources. Critique en environnement distribué.

S3.4 — Codes de statut HTTP

Les codes HTTP renseignent le client sur le résultat. À connaître :

Code	Signification	Quand l'utiliser
200	OK	Succès générique. GET réussi, PATCH/PUT qui renvoie la ressource.
201	Created	Création réussie. Réponse standard d'un POST.
204	No Content	Succès sans corps. Typiquement DELETE.
400	Bad Request	Requête mal formée par le client.
401	Unauthorized	Pas authentifié.
403	Forbidden	Authentifié mais pas autorisé.
404	Not Found	La ressource n'existe pas.
409	Conflict	Conflit avec l'état actuel.
422	Unprocessable Entity	Bien formée mais sémantiquement invalide.
500	Internal Server Error	Erreur serveur, pas de la faute du client.
503	Service Unavailable	Service temporairement indisponible.

Pièges classiques

- Renvoyer 200 alors qu'on a créé : doit être 201.
- Renvoyer 200 avec un corps d'erreur : si c'est une erreur, le code DOIT être 4xx ou 5xx.
- Confondre 401 (qui es-tu ?) et 403 (je sais qui tu es mais non).

S3.5 — Pagination, filtrage, tri

Toute collection qui peut grossir DOIT être paginée. Dès le premier jour.

Offset / Limit

```
GET /products?page=1&page_size=20
```

Réponse :

```
{
  "count": 1247,
  "next": "/products?page=2",
  "previous": null,
  "results": [ ... ]
}
```

- Simple, intuitif, permet de sauter à n'importe quelle page.
- Performance dégrade sur très gros volumes.

Cursor-based (Stripe, Twitter)

```
GET /products?cursor=xyz&limit=20
```

- Performance constante quel que soit le volume.
- Pas de doublons ni trous.

Filtrage, recherche, tri

```
GET /products?category=electronics
```

```
GET /products?search=casque+sans+fil
```

```
GET /products?ordering=-price
```

S3.6 — Le versioning

Trois stratégies

Par URI (recommandé)

```
GET /api/v1/products/42
```

- Lisible, debuggable. Stratégie la plus utilisée.

Par header

```
Accept: application/vnd.flexshop.v2+json
```

- Plus puriste mais invisible à l'œil nu.

Par query param (à éviter)

- Mélange filtrage et versioning.

Bonnes pratiques

- Garder l'ancienne version 6-12 mois après la nouvelle.
- Annoncer la dépréciation avec un header Deprecation.
- Ne pas créer de v2 pour un changement mineur.

S3.7 — Erreurs structurées

Renvoyer juste un code 400 ne suffit pas. Standard moderne : RFC 7807 « Problem Details for HTTP APIs ».

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://flexshop.com/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "detail": "The 'price' field must be positive",
  "errors": {
    "price": ["Must be > 0"]
  }
}
```

- Toujours un message lisible (title, detail).
- Détailler les champs en erreur si validation.
- Ne JAMAIS exposer de stack trace en production.

S3.8 — Documentation OpenAPI / Swagger

OpenAPI (anciennement Swagger) est LE standard. Fichier YAML/JSON décrivant toute l'API.

Pourquoi c'est essentiel

- Les autres équipes peuvent intégrer sans poser de questions.
- Swagger UI génère une interface NAVIGABLE et TESTABLE.
- On peut générer automatiquement des clients (Python, Java, TS).

Génération automatique

- Django + drf-spectacular : génère depuis le code DRF.
- Symfony + nelmio/api-doc-bundle : pareil pour Symfony.

S3.9 — Les anti-patterns à éviter

- Verbes dans les URIs : /createUser, /getOrder.
- Singulier au lieu de pluriel : /product/42.
- Mauvais code de statut : 200 OK + body { error }.
- Endpoint « god » : POST /actions avec body { action }.
- Réponses non paginées.
- Pas de versioning.
- Stack traces en production.

S3.10 — À retenir

- REST est un STYLE défini par Fielding en 2000. 6 contraintes.
- Penser RESSOURCES (noms au pluriel) et VERBES HTTP.
- Cinq verbes : GET, POST, PUT, PATCH, DELETE. GET/PUT/DELETE sont idempotents.
- Codes de statut : 2xx succès, 4xx faute client, 5xx faute serveur.
- Paginer TOUTE collection dès le jour 1.
- Versioning par URI recommandé. Garder l'ancienne version 6-12 mois.
- Documenter en OpenAPI + Swagger UI, généré depuis le code.

S4

TP : votre premier micro-service

Django · DRF · OpenAPI · Postman · Git

S4 — Objectifs du TP

Premier TP qui applique tout ce qu'on a vu ce matin (S1, S2, S3). Vous codez votre PREMIER micro-service.

- Initialiser un projet Django avec Django REST Framework.
- Modéliser une ressource métier avec contraintes.
- Écrire un serializer qui valide et transforme en JSON.
- Exposer une API REST CRUD via ViewSet + Router.
- Configurer pagination, filtrage, recherche.
- Générer une doc OpenAPI 3.0 + Swagger UI.
- Tester avec Postman et écrire 3 tests unitaires.
- Pousser sur Git avec un README clair.

S4.1 — Contexte du TP

Vous êtes développeur dans une équipe qui construit FlexShop en micro-services. On vous confie le PREMIER service : le Catalogue.

Caractéristiques d'un Product

- id : auto-généré.
- name : chaîne obligatoire, 1 à 200 caractères.
- description : texte long, optionnel.
- price : décimal positif ou nul.
- stock : entier positif ou nul, défaut 0.
- category : chaîne courte, optionnelle.
- created_at / updated_at : auto-gérés.

Endpoints attendus

- GET /api/v1/products/ : liste paginée.
- GET /api/v1/products/{id}/ : détail.
- POST /api/v1/products/ : crée (201).
- PUT /api/v1/products/{id}/ : remplace (200).
- PATCH /api/v1/products/{id}/ : modifie partiellement (200).
- DELETE /api/v1/products/{id}/ : supprime (204).

S4.2 — Les 10 étapes du TP

Suivez dans l'ordre, chaque étape s'appuie sur la précédente.

Étape 1 — Initialiser le projet (10 min)

```
mkdir catalog-service && cd catalog-service
python3 -m venv venv && source venv/bin/activate
pip install django djangorestframework drf-spectacular django-filter
pip freeze > requirements.txt
django-admin startproject catalog_project .
python manage.py startapp products
```

Étape 2 — Configurer settings.py (10 min)

```
INSTALLED_APPS = [
    # ... apps Django par défaut ...
    'rest_framework',
    'drf_spectacular',
    'django_filters',
    'products',
]

REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
    'DEFAULT_PAGINATION_CLASS':
        'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
}
```

Étape 3 — Le modèle Product (15 min)

Dans products/models.py :

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=200)
    description = models.TextField(blank=True, default='')
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    category = models.CharField(max_length=50, blank=True, default='')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

```
class Meta:
    ordering = ['-created_at']
python manage.py makemigrations
python manage.py migrate
```

🔗 Discussion technique : Decimal ou centimes ?

Deux approches valides pour stocker des montants :

DecimalField (ce TP) : on stocke 89.90 directement. Lisible dans le JSON.

IntegerField en centimes : on stocke 8990 et on convertit à l'affichage. C'est l'approche de Stripe, PayPal, Square.

Ne JAMAIS : FloatField. Arrondis binaires catastrophiques.

Étape 4 — Le serializer (10 min)

```
from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ['id', 'name', 'description', 'price',
                  'stock', 'category', 'created_at', 'updated_at']
        read_only_fields = ['id', 'created_at', 'updated_at']

    def validate_price(self, value):
        if value < 0:
            raise serializers.ValidationError('Le prix ne peut pas être négatif.')
        return value
```

Étape 5 — Le ViewSet (15 min)

```
from rest_framework import viewsets, filters
from django_filters.rest_framework import DjangoFilterBackend
from .models import Product
from .serializers import ProductSerializer

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    filter_backends = [DjangoFilterBackend, filters.SearchFilter,
                      filters.OrderingFilter]
    filterset_fields = ['category']
    search_fields = ['name', 'description']
    ordering_fields = ['price', 'created_at', 'stock']
```

ModelViewSet expose AUTOMATIQUEMENT list, retrieve, create, update, partial_update, destroy. C'est ÇA la puissance de DRF.

Étape 6 — Router + URLs (10 min)

products/urls.py :

```
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

router = DefaultRouter()
router.register(r'products', ProductViewSet, basename='product')

urlpatterns = router.urls

catalog_project/urls.py :
```

```
from drf_spectacular.views import (SpectacularAPIView, SpectacularSwaggerView)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/v1/', include('products.urls')),
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema')),
]
```

Étape 7 — Lancer (5 min)

```
python manage.py runserver
# Naviguer vers : http://localhost:8000/api/v1/products/
# Doc Swagger : http://localhost:8000/api/docs/
```

Étape 8 — Tests unitaires (15 min)

```
from rest_framework.test import APITestCase
from .models import Product

class ProductAPITests(APITestCase):
    def setUp(self):
        self.product = Product.objects.create(
            name='Casque XYZ', price=89.90, stock=10,
            category='electronics')

    def test_list_products(self):
        response = self.client.get('/api/v1/products/')
        self.assertEqual(response.status_code, 200)

    def test_create_product(self):
        data = {'name': 'Souris', 'price': '29.99', 'stock': 50}
        response = self.client.post('/api/v1/products/', data, format='json')
```

```

        self.assertEqual(response.status_code, 201)

def test_get_nonexistent_returns_404(self):
    response = self.client.get('/api/v1/products/99999/')
    self.assertEqual(response.status_code, 404)
python manage.py test products

```

Étape 9 — Collection Postman (10 min)

- GET /api/v1/products/
- GET /api/v1/products/1/
- POST /api/v1/products/ + body JSON
- PATCH /api/v1/products/1/ + body JSON (price)
- DELETE /api/v1/products/1/
- GET /api/v1/products/?search=casque
- GET /api/v1/products/?ordering=-price

Astuce de pro : Postman peut importer le schéma OpenAPI ! File → Import → URL
<http://localhost:8000/api/schema/>.

Étape 10 — README + Git (5 min)

README.md à la racine :

```

# Catalog Service

Premier micro-service du projet FlexShop.

## Démarrer
```bash
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

## Endpoints
- Doc Swagger : http://localhost:8000/api/docs/
- API : http://localhost:8000/api/v1/products/

```

.gitignore (CRITIQUE) :

```

venv/
__pycache__/
*.pyc
db.sqlite3
.env
git init && git add . && git commit -m 'Initial catalog'

```



```
git remote add origin <URL_REPO>  
git push -u origin main
```

S4.3 — Pièges classiques à éviter

Piège 1 — `ModuleNotFoundError`

Cause : venv pas activé. Vérifier que le prompt affiche (venv).

Piège 2 — App 'products' isn't installed

Oubli d'ajouter 'products' dans `INSTALLED_APPS`.

Piège 3 — Modèle changé sans `makemigrations`

Après TOUTE modification de `models.py` : `makemigrations` puis `migrate`.

Piège 4 — POST renvoie 415 `Unsupported Media Type`

Postman : Body → raw → JSON (pas Text).

Piège 5 — POST renvoie 400 mais on ne sait pas pourquoi

Lire le BODY de la réponse, DRF dit précisément quel champ pose problème.

Piège 6 — Filtrage ne marche pas

Vérifier : `django_filters` dans `INSTALLED_APPS` + `DjangoFilterBackend` + `filterset_fields`.

Piège 7 — `db.sqlite3` ou venv/ poussés sur Git

Vérifier le `.gitignore` AVANT le premier git add.

Piège 8 — `FloatField` pour le prix

Faute éliminatoire. `DecimalField` (ce TP) ou `IntegerField` en centimes.

S4.4 — Livrables et évaluation

À rendre dans 7 jours

- Lien vers le dépôt Git.
- Capture d'écran de Swagger UI fonctionnel.
- Collection Postman exportée (.json).
- Compte rendu PDF 2 pages.

| Critère | Pts | Détail |
|-----------------------|-----|---|
| Code fonctionnel | 6 | Tous les endpoints CRUD répondent. Pagination, filtrage OK. |
| Conformité REST | 3 | URIs au pluriel sans verbes, codes de statut corrects. |
| Documentation OpenAPI | 3 | Swagger UI accessible et complet. |
| Tests unitaires | 2 | Au moins 3 tests qui passent. |
| Qualité projet | 1 | README clair, .gitignore correct, requirements.txt. |
| TOTAL | 15 | Sans bonus |

Bonus possibles (jusqu'à +2 pts)

- Endpoint custom avec @action.
- Validation métier customisée pertinente.
- Prix stocké en centimes (approche fintech).

Malus

- db.sqlite3 ou venv/ committé : -1 pt.
- URI avec verbes ou singuliers : -2 pts.
- FloatField : -2 pts.
- Pas de README : -1 pt.

S4.5 — À retenir

- DRF + ModelViewSet + Router = API REST CRUD complète en moins de 20 lignes utiles.
- Ordre : modèle → makemigrations → migrate → serializer → view → URL.
- Le serializer valide + transforme en JSON. Il ne sauve pas.
- PAGE_SIZE dans settings = pagination par défaut. Structure : { count, next, previous, results }.
- drf-spectacular génère la doc OpenAPI. Swagger UI sur /api/docs/.
- Tester avec APITestCase. self.client.get/post/patch.
- Montants : JAMAIS FloatField. DecimalField ou IntegerField en centimes.

Devoir global de la journée

Un compte rendu unique qui couvre toute la journée, à rendre dans 7 jours.

Format

- 4 à 5 pages PDF maximum.
- Une seule remise pour toute la journée.
- À déposer sur la plateforme de l'école avant la date limite.

Structure demandée

Section 1 — Découpage en services (issu de S1 et S2)

Choisissez un système parmi : une plateforme de livraison de repas, une banque en ligne, ou une plateforme MOOC.

- Proposez un découpage en 5 à 8 services.
- Justifiez chaque service par sa capacité métier.
- Identifiez les bounded contexts du DDD.
- Vérifiez que vous évitez les anti-patterns vus en S2.

Section 2 — Conception API REST (issu de S3)

Choisissez l'UN de vos services de la section 1.

- Listez les endpoints REST (URI + verbe HTTP + code de retour).
- Donnez un exemple de payload JSON pour création.
- Précisez votre stratégie de pagination, filtrage, versioning.

Section 3 — Retour sur le TP (issu de S4)

- Choix techniques notables (Decimal ou centimes ? Filtrage ?).
- Difficultés rencontrées et solutions trouvées.
- Ce qui resterait à améliorer pour aller en production.

Section 4 — Lien Git

Lien vers le dépôt Git de votre service Catalogue (du TP S4).

Évaluation

- Section 1 : 6 points (qualité du découpage, justifications).
- Section 2 : 4 points (conformité REST, complétude).
- Section 3 : 3 points (réflexion personnelle).
- Section 4 + qualité globale : 2 points.

Pour aller plus loin

- Article fondateur : « Microservices » par Lewis & Fowler (2014) sur martinfowler.com.
- « Building Microservices » de Sam Newman (O'Reilly, 2021).
- « Domain-Driven Design » d'Eric Evans (2003).
- « The Design of Web APIs » d'Arnaud Lauret (Manning, 2019).
- « Team Topologies » de Skelton & Pais (2019).
- Documentation Django REST Framework : www.django-rest-framework.org.